

Deep Learning and Its Application

Week 4: Practicals

1. Implement the following Mean Squared Error (MSE) loss function in Python

$$MSE = \frac{\sum_{i=1}^m (y_i - \hat{y}_i)^2}{m}$$

Given the true values $y_i = [0.000, 0.166, 0.333]$, and predicted values $\hat{y}_i = [0.000, 0.254, 0.998]$, use the implemented function to calculate error.

Answer:

```
import numpy as np

y_hat = np.array([0.000, 0.166, 0.333])
y_true = np.array([0.000, 0.254, 0.998])

def rmse(predictions, targets):
    differences = predictions - targets
    differences_squared = differences ** 2
    mean_of_differences_squared = differences_squared.mean()
    rmse_val = np.sqrt(mean_of_differences_squared)
    return rmse_val

print("d is: " + str(["%.8f" % elem for elem in y_hat]))
print("p is: " + str(["%.8f" % elem for elem in y_true]))
rmse_val = rmse(y_hat, y_true)
print("rms error is: " + str(rmse_val))
```

2. Implement the following Mean Absolute Error (MAE) loss function in Python

$$MAE = \frac{\sum_{i=1}^m |y_i - \hat{y}_i|}{m}$$

Given the true values $y_i = [0.000, 0.166, 0.333]$, and predicted values $\hat{y}_i = [0.000, 0.254, 0.998]$, use the implemented function to calculate error.

Answer:

```
import numpy as np

y_hat = np.array([0.000, 0.166, 0.333])
y_true = np.array([0.000, 0.254, 0.998])

print("d is: " + str(["%.8f" % elem for elem in y_hat]))
print("p is: " + str(["%.8f" % elem for elem in y_true]))

def mae(predictions, targets):
    differences = predictions - targets
    print(differences)
    absolute_differences = np.absolute(differences)
    print(absolute_differences)
    mean_absolute_differences = absolute_differences.mean()
    return mean_absolute_differences
```

```
mae_val = mae(y_hat, y_true)
print ("mae error is: " + str(mae_val))
```

3. Implement the following Cross Entropy loss function in Python

$$\text{Cross Entropy} = - \sum \sum_{i=1}^k y_i \log \hat{y}_i$$

Given the true values $y = [[0,0,0,1], [0,0,0,1]]$,
 predicted values $\hat{y} = [[0.25,0.25,0.25,0.25], [0.01,0.01,0.01,0.96]]$ and k is the number of classes, use the implemented function to calculate error.

Answer:

```
import numpy as np
predictions = np.array([[0.25,0.25,0.25,0.25],
                        [0.01,0.01,0.01,0.96]])
targets = np.array([[0,0,0,1],
                    [0,0,0,1]])
def cross_entropy(predictions, targets, epsilon=1e-10):
    predictions = np.clip(predictions, epsilon, 1. - epsilon)
    N = predictions.shape[0]
    print(N)
    ce_loss = -np.sum(np.sum(targets * np.log(predictions + 1e-5)))/N
    return ce_loss
cross_entropy_loss = cross_entropy(predictions, targets)
print ("Cross entropy loss is: " + str(cross_entropy_loss))
```

4. Given the following fragment code

```
from numpy import argmax
from pandas import read_csv
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

# load the dataset
path = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv'
df = read_csv(path, header=None)
# split into input and output columns
X, y = df.values[:, :-1], df.values[:, -1]
# ensure all data are floating point values
X = X.astype('float32')
# encode strings to integer
y = LabelEncoder().fit_transform(y)
# split into train and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
# determine the number of input features
n_features = X_train.shape[1]
.
.
.

row2 = np.array([[5.1,3.5,1.4,0.2]])
```

```
yhat = model.predict([row2])
print('Predicted: %s (class=%d)' % (yhat, argmax(yhat)))
```

Use Functional model to develop a multiclass classification model using Iris dataset, display

- a) Training accuracy
- b) Prediction results

Answer:

```
from numpy import argmax
from pandas import read_csv
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow import keras
import numpy as np

# load the dataset
path = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv'
df = read_csv(path, header=None)
# split into input and output columns
X, y = df.values[:, :-1], df.values[:, -1]
# ensure all data are floating point values
X = X.astype('float32')
# encode strings to integer
y = LabelEncoder().fit_transform(y)
# split into train and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
# determine the number of input features
n_features = X_train.shape[1]
# define model

# define model by Functional API
x_in = Input(shape=(n_features,))
x2 = Dense(10, activation='relu', kernel_initializer='he_normal')(x_in)
x3 = Dense(10, activation='relu', kernel_initializer='he_normal')(x2)
x_out = Dense(3, activation='softmax')(x3)
# define the model
model = keras.Model(inputs=x_in, outputs=x_out)

# compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
# fit the model
model.fit(X_train, y_train, epochs=150, batch_size=32, verbose=0)
# evaluate the model
loss, acc = model.evaluate(X_test, y_test, verbose=0)

print('Test Accuracy: %.3f' % acc)
# make a prediction
row1 = [5.1, 3.5, 1.4, 0.2]

row2 = np.array([5.1, 3.5, 1.4, 0.2])

yhat = model.predict([row2])
print('Predicted: %s (class=%d)' % (yhat, argmax(yhat)))
```

5. Given the following fragment code

```
from numpy import sqrt
from pandas import read_csv
from sklearn.model_selection import train_test_split
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

# load the dataset
path =
'https://raw.githubusercontent.com/jbrownlee/Datasets/master/housing.csv'
df = read_csv(path, header=None)
# split into input and output columns
X, y = df.values[:, :-1], df.values[:, -1]
# split into train and test datasets
X_train, X_test, y_train, y_test =
.
.
.

row2 =
np.array([[0.00632,18.00,2.310,0,0.5380,6.5750,65.20,4.0900,1,296.0,15.30,396.9
0,4.98]])
yhat = model.predict([row2])
print('Predicted: %.3f' % yhat)
```

Use the **Functional** model to develop a regression model using Boston Housing dataset, display

- c) Squared sum error
- d) Prediction results

Answer:

```
from numpy import sqrt
from pandas import read_csv
from sklearn.model_selection import train_test_split
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Input
from tensorflow import keras
import numpy as np

from sklearn.datasets import make_moons, make_circles, make_classification

# load the dataset
path = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/housing.csv'
df = read_csv(path, header=None)
# split into input and output columns
X, y = df.values[:, :-1], df.values[:, -1]
# split into train and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
# determine the number of input features
n_features = X_train.shape[1]
# define model

x_in = Input(shape=(n_features,))
x = Dense(10, activation='relu', kernel_initializer='he_normal')(x_in)
x_out = Dense(1)(x)
```

```

# define the model
model = keras.Model(inputs=x_in, outputs=x_out)

# compile the model
model.compile(optimizer='adam', loss='mse')
# fit the model
model.fit(X_train, y_train, epochs=150, batch_size=32, verbose=0)
# evaluate the model
error = model.evaluate(X_test, y_test, verbose=0)
print('MSE: %.3f, RMSE: %.3f' % (error, sqrt(error)))
# make a prediction
row1 = [0.00632, 18.00, 2.310, 0, 0.5380, 6.5750, 65.20, 4.0900, 1, 296.0, 15.30, 396.90, 4.98]
row2 =
np.array([[0.00632, 18.00, 2.310, 0, 0.5380, 6.5750, 65.20, 4.0900, 1, 296.0, 15.30, 396.90, 4.98]])
yhat = model.predict([row2])
print('Predicted: %.3f' % yhat)

```

6. Read Chapter 3: Improving the way neural networks learn at the following website

<http://neuralnetworksanddeeplearning.com/chap3.html>